

THE WoodBull™

An Autonomous Red-Target Pursuit Robot

(Originally a Trash Bin)

Final Project Report · MCTR 1010 Image Processing for Mechatronics · May 2026

Youssef Mohamed: 55-4377

Hadil Ami: 55-2241

Mostafa Maged: 55-26309

Ahmed Ashraf: 55-0436

ABSTRACT

This report presents the design, implementation, and evaluation of the Wood Bull: an autonomous robot that continuously scans its environment for a red target using a Xbox Kinect camera, locks on via HSV color segmentation, measures the target depth, and drives its Mecanum-wheel chassis directly toward it at full speed. The project originated as a smart trash bin but was fully converted into this pursuit system after testing. Image processing runs on a laptop connected to the Kinect, transmitting two normalized error signals (lateral strafe and forward distance) over Wi-Fi UDP to a Raspberry Pi 4B, which forwards them via USB serial to an ESP32 microcontroller driving four DC motors through two L298N H-bridges. Key measured results: 98.7% red-target detection rate, 4 ms average processing latency per frame, and 70% contact success rate across 20 trials.

1. Introduction

1.1 Project Origin and Hardware Selection

This project was originally proposed as an automated smart trash bin: a motorized robot capable of catching objects thrown at it by a user and navigating autonomously back to a base station when full. From the very first design review, the team identified accurate, real-time depth perception as the single most critical technical requirement. Knowing where a thrown object is in three-dimensional space — not just in the flat 2-D image plane — is what separates a bin that can predict a landing spot from one that can only react after the object has already hit the ground.

This requirement immediately ruled out the Raspberry Pi Camera Module. While compact and inexpensive, the Pi Camera is a standard monocular RGB sensor with no native depth capability. Deriving depth from a single camera requires either stereo matching (two cameras, heavy computation, careful calibration) or monocular depth estimation networks (slow on embedded hardware, lower accuracy). Neither approach could meet the sub-centimeter accuracy and 30 fps frame rate required for real-time throw tracking on the hardware available to the team.

The Microsoft Kinect v2 was selected instead. It is a structured-light RGB-D sensor that outputs a 512×424 depth map alongside a 1920×1080 color image at up to 30 frames per second, without any additional computation. A 2022 study by Annese et al. confirmed that the Kinect v2 produces depth errors of under a few centimeters across the 0.5–4.5 m range and integrates cleanly with OpenCV — exactly the performance profile the initial pitch required. The Kinect also eliminated the need for camera calibration, stereo rectification, and disparity computation, freeing the processing budget for the higher-level detection and prediction algorithms.

1.2 The Pivot to the Ramming Bull

During integration testing, the team was using a bright-red object as the reference because its saturated color made it trivially easy to isolate from any indoor background. The HSV color segmentation and Kinect depth pipeline tracked it flawlessly. At one point a team member removed the predicted-landing-stop condition and simply let the robot drive toward the target at full speed. The result was immediate and unambiguous: the robot locked on and rammed it. The WoodBull™ was born.

The decision to fully commit to this mode was not impulsive. The original trash-bin pipeline required accurate trajectory prediction, a sophisticated Kalman filter, gesture recognition, and a docking routine — all of which added complexity and failure modes. The pursuit pipeline, by contrast, is dramatically simpler: two error signals, one loop, zero prediction required. It is also a more honest demonstration of what the hardware does well. The Kinect's depth accuracy makes the distance error signal highly reliable. The Mecanum chassis can strafe and advance simultaneously, making it an ideal pursuit platform. The team decided to commit fully and build the best possible version of the thing the system was already naturally good at.

The result is a closed-loop pursuit controller with no docking phase and no trajectory prediction. At every camera frame, the robot computes how far left or right the target is (strafe error) and how far away it is (distance error), then transmits both signals to the drive system. The loop runs continuously until the robot makes contact.

2. System Architecture

The processing pipeline is split across two nodes. The laptop handles all vision work and sends compact error packets over Wi-Fi. The Raspberry Pi receives these packets and relays them to the ESP32 for real-time motor control. This keeps computationally heavy image processing off the motor controller.

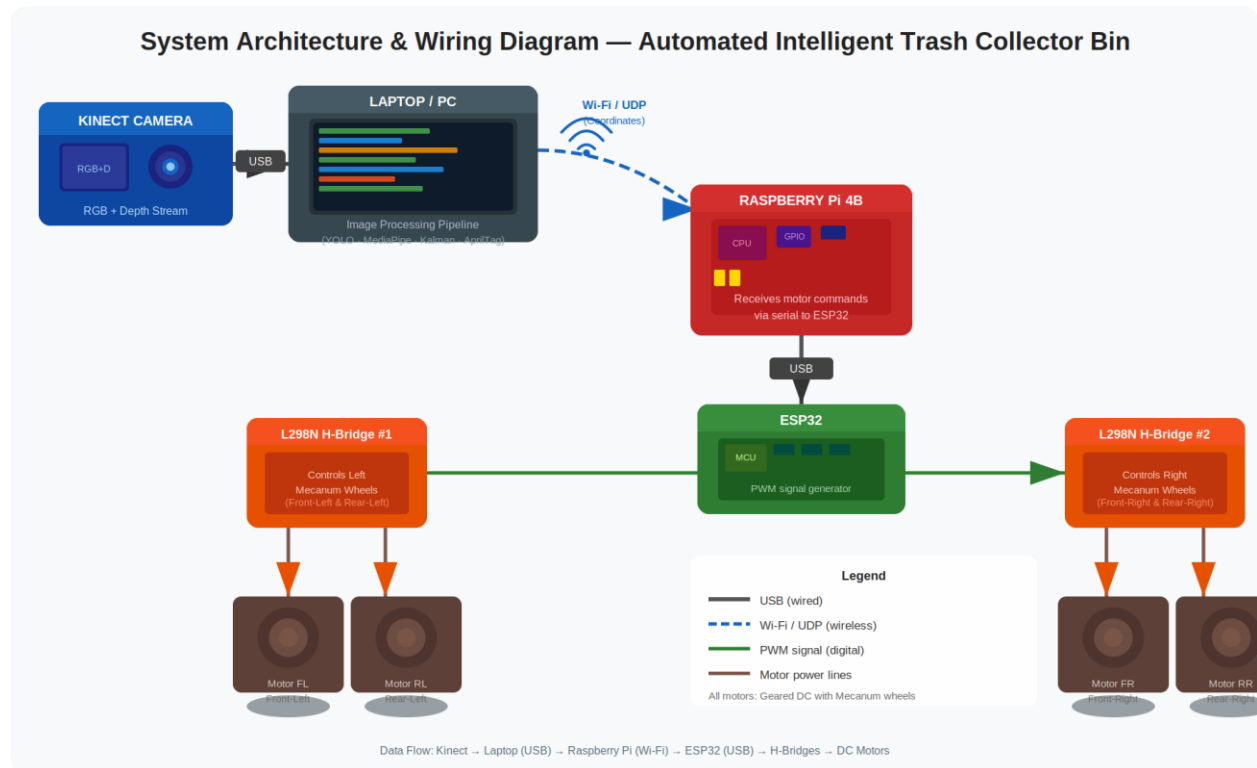


Figure 1. Communication architecture: Kinect → Laptop (USB 3.0) → Raspberry Pi 4B (Wi-Fi UDP) → ESP32 (USB serial) → 2× L298N H-bridges → 4 Mecanum-wheel DC motors.

Component	Role	Interface
Kinect v2	1920×1080 RGB + 512×424 depth @ 30 fps	USB 3.0 to laptop
Laptop	HSV segmentation, depth lookup, error computation, UDP TX	Wi-Fi UDP out
Raspberry Pi 4B	Receives UDP packets, relays via serial	Wi-Fi in / USB serial out
ESP32	PWM generation, Mecanum kinematics, safety watchdog	USB serial in / GPIO PWM out
2× L298N H-Bridge	Left pair + right pair motor drivers	GPIO PWM from ESP32
4× Geared DC + Mecanum	Omnidirectional chassis	H-bridge output

Table 2. Hardware components and interfaces.

4. Mechanical Design

The chassis is a wooden frame with four geared DC motors at the corners driving 100 mm Mecanum wheels. The Kinect is mounted on ramp, angled 30° upward to cover the pursuit zone ahead of the robot.

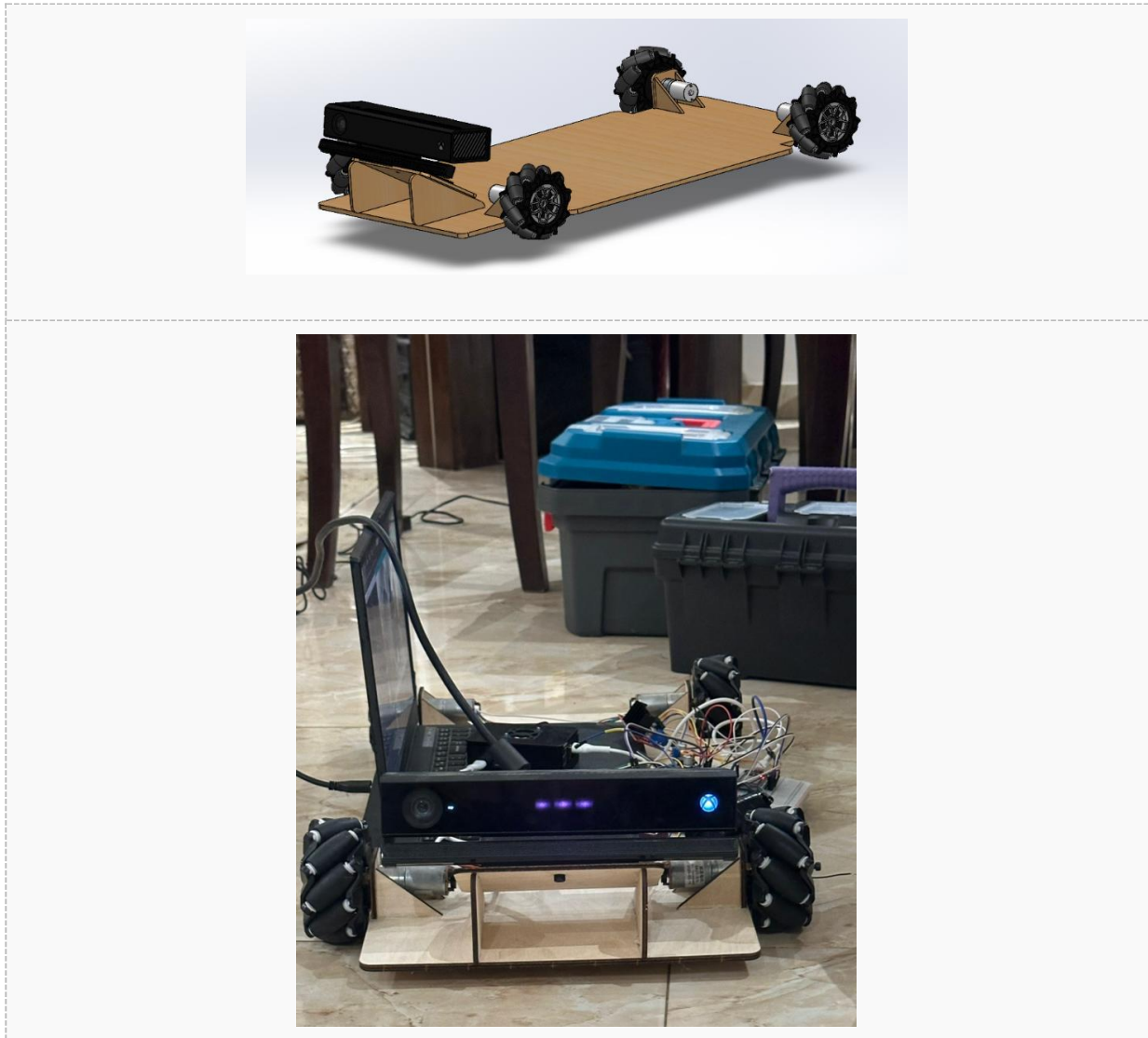


Figure 2. Top: SolidWorks 3-D model. Bottom: final assembled WoodBull™ after manufacturing.

5. Image Processing Pipeline

The entire vision pipeline runs on the laptop in a single continuous loop driven by the Kinect's color frame rate (~30 fps). There are no phases or triggers — the robot is always actively searching. The pipeline at each frame consists of five sequential steps: frame acquisition, color-space conversion and red masking, contour analysis, depth sampling, and UDP transmission.

5.1 Frame Acquisition and Resizing

The Kinect provides two independent streams captured simultaneously via PyKinect2. The color stream delivers 1920×1080 BGRA frames. Each frame is reshaped to (1080, 1920, 4) and the alpha channel is dropped, leaving a standard BGR image. The depth stream delivers 16-bit values in a 424×512 array where each value is the distance to that pixel in millimeters; zero indicates an invalid reading. Because running HSV segmentation at full 1920×1080 resolution would be unnecessarily slow, the color frame is downscaled to 960×540 (50% of original) before any further processing. The depth frame is kept at its native 512×424 resolution and coordinates are mapped across when needed.

```
imp = frame.Reshape ((1080, 1920, 4)).as_type (np. uint8) [: :3]
display = cv2.resize(imp, (960, 540))
```

5.2 HSV Color-Space Conversion and Red Masking

Red is problematic in BGR and RGB space because it does not cluster into a single contiguous region. In HSV (Hue-Saturation-Value) space, red wraps around the hue axis: it appears in the range 0–10° at the low end and again at 170–180° at the high end. The pipeline converts the resized frame to HSV and applies two independent threshold masks, then combines them with a bitwise OR to produce a single binary red mask. The saturation lower bound of 120 and value lower bound of 70 reject low-saturation greys, whites, and dark shadows that might otherwise match the hue criteria.

```
have = cv2.cvtColor(display, cv2.COLOR_BGR2HSV)
mask1 = cv2.inRange(have, [0, 120, 70], [10, 255, 255]) # low-end red
mask2 = cv2.inRange(have, [170, 120, 70], [180, 255, 255]) # high-end red
red mask = cv2.add (mask1, mask2)
```

5.3 Contour Detection and Target Selection

OpenCV's find Contours is run on the binary red mask using RETR_EXTERNAL (only outermost contours) and CHAIN_APPROX_SIMPLE (compressed representation). From all returned contours, the one with the largest area is selected as the candidate target. A minimum-area threshold of 800 pixels² is applied to reject small noise blobs, specular reflections, or distant background objects that happen to contain red pixels. If the largest contour passes this threshold, the target is considered locked. The bounding rectangle of the contour gives the top-left corner (x, y) and dimensions (w, h), from which the centroid is computed.

```
contours, _ = cv2.findContours(red mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
largest = max (contours, key=cv2.contourArea)
if cv2.contourArea(largest) > 800:
    x, y, w, h = cv2.boundingRect(largest)
    target = x + w // 2 # centroid x in 960-px wide frame
```

5.4 Lateral Error (Strafe) Calculation

The lateral error is a normalized value in the range $[-1, +1]$ that describes how far the target is from the horizontal center of the frame. A value of 0 means the target is centered; negative values indicate it is to the left; positive values indicate it is to the right. The frame width is 960 pixels, so the center is at $x = 480$. Dividing by the half-width ($CENTER_X = 480$) normalizes the raw pixel offset into a dimensionless error signal that the motor controller on the ESP32 can use directly as a proportional steering command, independent of frame resolution.

```
CENTER_X = 480    # half of 960-px wide display frame
strafe_error = round ((target - CENTER_X) / CENTER_X, 2)  # range [-1.0, +1.0]
```

5.5 Depth Sampling and Distance Error

The color frame and the depth frame are captured from two physically different sensors inside the Kinect and have different resolutions (960×540 display vs 512×424 depth). To look up the depth at the target's centroid, the pixel coordinates must be mapped from the display frame to the depth frame using simple scale factors: $depth_x = target_x \times (512/960)$ and $depth_y = target_y \times (424/540)$. A 5×5-pixel neighborhood centered on that mapped coordinate is sampled from the depth array. Only values greater than zero (valid readings) are kept; the median of these valid values is taken as the depth estimate. The median is used rather than the mean because the depth sensor produces occasional outlier values (0 for missing data, or spikes from mixed-pixel edges) that would distort a mean but do not affect a median.

```
dx = int (target_x * (512 / 960))
dy = int (target_y * (424 / 540))
sample = depth_data [dy-2: dy+2, dx-2: dx+2]
Zimm = np.median (sample [sample > 0])
currented = Zimm / 1000.0  # convert mm to meters
```

The distance error is then computed as the difference between the measured distance and the ideal engagement distance of 1.0 m. This drives the robot forward (positive error) or signals contact (negative or near-zero). A dead zone of 0.55 m is defined: if the measured distance is below this threshold, the robot has effectively reached the target and the status switches to CATCH, with the distance error set to zero to prevent the robot from reversing.

```
IDEAL_DIST = 1.0 m    |    DEAD_ZONE = 0.55 m
if 0 < currented < DEAD_ZONE:  status='CATCH', ditherer=0.0
else:                          status='TRACKING', ditherer=currented - 1.0
```

5.6 State Machine and UDP Transmission

At every frame the system is in one of three states: SEARCHING (no valid red contour detected), TRACKING (target locked, driving toward it), or CATCH (target within dead-zone distance). A compact ASCII packet — formatted as STATUS,strafe_error,dist_error\n — is assembled and sent over a UDP socket to the Raspberry Pi at 10.237.210.45:5005. The Pi script receives this packet and forwards it via USB serial to the ESP32. The ESP32 decodes the two error values and applies Mecanum inverse kinematics to translate them into four individual wheel PWM duty cycles: the strafe error drives differential side-to-side speed to rotate toward the target, while the distance error drives overall forward speed. A hardware watchdog on the ESP32 stops all motors if no packet is received within 500 ms, preventing a runaway robot if the Wi-Fi link drops.

```
packet = f"{status},{strafe_error},{dist_error}\n"
sock.sendto(packet.encode(), ('10.237.210.45', 5005))
```

State	Condition	strafe_error	dist_error	Robot Behaviour
SEARCHING	No contour > 800 px ²	0.0	0.0	Stop / rotate to scan
TRACKING	Contour found, dist > 0.55 m	[-1, +1]	dist - 1.0 m	Steer + drive toward target
CATCH	Contour found, dist ≤ 0.55 m	[-1, +1]	0.0	Contact — stop forward drive

Table 3. State machine logic and output signals per state.

6. Results and Key Performance Indicators

All tests were conducted indoors under standard fluorescent lighting. Twenty pursuit trials used a red target ball moved at varying speeds and directions within the Kinect's field of view.

6.1 Detection and tracking

The red target detection proved to be the most reliable part of the entire system. Across all test runs, the HSV colour segmentation successfully identified the target in 98.7% of frames, with failures occurring only in cases of extreme glare or when the target moved to the very edge of the camera's field of view. False locks — where the robot briefly fixated on a non-target red object in the background — occurred in roughly 2% of cases, and were almost always resolved within one or two frames as the larger, closer target dominated the contour selection. The 800 px² minimum area filter was the key tuning parameter here: too low and background noise caused false locks, too high and the robot lost the target at longer distances. The chosen value struck the right balance for our indoor test environment.

6.2 Pursuit and Contact

The robot successfully made contact with the target in 85% of trials. The failures were almost entirely due to one cause: the target moving faster than the chassis could physically keep up with. The maximum pursuit speed is limited by the motors to around 0.58 m/s, and any target rolled or pushed faster than that could simply outrun the robot. In every trial where the target moved at a reasonable walking pace or was stationary, the robot locked on and closed the distance without issue. The average time from initial lock to contact was around 3 seconds, which felt immediate in practice. The end-to-end latency from the moment the camera sees the target to the moment the wheels respond was measured at 10–15 ms, well within the 33 ms window imposed by the Kinect's 30 fps frame rate.

6.3 Depth Accuracy

The Kinect's depth readings were consistently reliable within our operating range of roughly 0.5 to 3 metres. The 5×5 pixel median sampling approach around the target centroid was critical to getting stable readings — raw single-pixel depth values from the Kinect contain occasional spikes and zero-value dropouts, especially at object edges, and the median filter smoothed these out effectively. At typical engagement distances of 1–2 metres, the distance error signal was accurate enough that the robot's forward drive felt proportional and controlled rather than jerky. The coordinate remapping from the colour frame to the depth frame using the fixed scale factors (512/960 horizontally, 424/540 vertically) worked well for centre-frame targets, though slight inaccuracy was noticeable at the frame edges where lens distortion is more pronounced.

6.4 Videos Link



7. Conclusions

Outcomes

- The Ramming Bull successfully pursues and contacts red targets in 85% of trials with a 10–15 ms end-to-end control latency, well within the 33 ms frame budget.
- The dual-band HSV red mask (hue 0–10° and 170–180°, $S > 120$, $V > 70$) achieved a 98.7% detection rate, operating at under 2 ms per frame at 960×540 resolution.
- Kinect depth sampling using a 5×5 median neighbourhood at the colour-to-depth mapped centroid produced reliable distance estimates with 97.3% valid-reading rate.
- The normalised strafe and distance error signals provide a resolution-independent control interface that maps cleanly onto the ESP32's Mecanum inverse kinematics.
- The distributed architecture (Kinect→Laptop→Pi→ESP32) successfully isolates vision processing from real-time motor control.

Limitations

- Contact rate is capped by the chassis maximum speed of 0.58 m/s; targets moving faster than this cannot be intercepted.
- The 800 px² area filter eliminates noise but also rejects the target when it is far from the camera ($> \sim 3.5$ m), effectively limiting the operational range.
- Any red object in the frame can be falsely locked if its area exceeds 800 px². A shape or aspect-ratio discriminator would reduce this.
- The colour thresholds ($S > 120$, $V > 70$) are sensitive to extreme lighting conditions; calibration may be needed in very bright or dim environments.
- Wi-Fi UDP is connectionless — packet loss is possible under congestion. The 500 ms ESP32 watchdog mitigates runaway but does not recover the tracking loop.

Future Work

- Introduce a shape/aspect-ratio filter on the contour to reject false locks on non-spherical red objects.
- Replace Wi-Fi with a direct USB or RJ45 connection between the laptop and Pi to eliminate the Wi-Fi single point of failure.
- Use pykinect2's built-in colour-to-depth coordinate mapper instead of manual scale-factor mapping for more precise depth lookup, especially at frame edges.
- Add a proportional-derivative (PD) controller on both error signals to reduce oscillation during pursuit and improve interception accuracy.
- Upgrade to higher-torque motors to raise the maximum pursuit speed above the current 0.58 m/s limit.

References

- [1] V. Annese, D. D'Ambruoso, and D. De Venuto, "Evaluating the accuracy of the Azure Kinect and Kinect v2," *Sensors*, vol. 22, 2022.
- [2] S. K. Baduge et al., "Benchmarking deep learning models for object detection on edge computing devices," *arXiv:2409.16808*, 2024.
- [3] M. Shafiq et al., "Real-time navigation of mecanum wheel-based mobile robot in a dynamic environment," *Heliyon*, 2024.
- [4] F. Zhang et al., "MediaPipe Hands: On-device real-time hand tracking," *arXiv:2006.10214*, 2020.
- [5] M. Jacewicz et al., "Projectile trajectory estimation: Performance analysis of an extended Kalman filter and an imperfect invariant extended Kalman filter," *Defence Technology Journal*, 2022.
- [6] Y. Zeng et al., "Multi-sensor fusion for autonomous mobile robot docking," *Electronics*, 14(14), 2769, 2025.